

and later writing the resulting video, the encoder spends all its time applying the video codec to the raw data, easily consuming 100% of the processor. The video encoder does not have any strong time constraints on when it runs—if it started running now or in half a second, the user could not tell and would not care. Of course, the sooner it finishes the better, but latency is not a primary concern.

In this scenario, ideally the scheduler gives the text editor a larger proportion of the available processor than the video encoder, because the text editor is interactive. We have two goals for the text editor. First, we want it to have a large amount of processor time available to it; not because it needs a lot of processor (it does not) but because we want it to always have processor time available the moment it needs it. Second, we want the text editor to preempt the video encoder the moment it wakes up (say, when the user presses a key). This can ensure the text editor has good *interactive performance* and is responsive to user input. On most operating systems, these goals are accomplished (if at all) by giving the text editor a higher priority and larger timeslice than the video encoder. Advanced operating systems do this automatically, by detecting that the text editor is interactive. Linux achieves these goals too, but by different means. Instead of assigning the text editor a specific priority and timeslice, it guarantees the text editor a specific proportion of the processor. If the video encoder and text editor are the only running processes and both are at the same nice level, this proportion would be 50%—each would be guaranteed half of the processor's time. Because the text editor spends most of its time blocked, waiting for user key presses, it does not use anywhere near 50% of the processor. Conversely, the video encoder is free to use *more* than its allotted 50%, enabling it to finish the encoding quickly.

The crucial concept is what happens when the text editor wakes up. Our primary goal is to ensure it runs immediately upon user input. In this case, when the editor wakes up, CFS notes that it is allotted 50% of the processor but has used considerably less. Specifically, CFS determines that the text editor has run for *less time* than the video encoder. Attempting to give all processes a *fair share* of the processor, it then preempts the video encoder and enables the text editor to run. The text editor runs, quickly processes the user's key press, and again sleeps, waiting for more input. As the text editor has not consumed its allotted 50%, we continue in this manner, with CFS always enabling the text editor to run when it wants and the video encoder to run the rest of the time.

The Linux Scheduling Algorithm

In the previous sections, we discussed process scheduling in the abstract, with only occasional mention of how Linux applies a given concept to reality. With the foundation of scheduling now built, we can dive into Linux's process scheduler.

Scheduler Classes

The Linux scheduler is modular, enabling different algorithms to schedule different types of processes. This modularity is called *scheduler classes*. Scheduler classes enable different, pluggable algorithms to coexist, scheduling their own types of processes. Each scheduler

class has a priority. The base scheduler code, which is defined in `kernel/sched.c`, iterates over each scheduler class in order of priority. The highest priority scheduler class that has a runnable process wins, selecting who runs next.

The Completely Fair Scheduler (CFS) is the registered scheduler class for normal processes, called `SCHED_NORMAL` in Linux (and `SCHED_OTHER` in POSIX). CFS is defined in `kernel/sched_fair.c`. The rest of this section discusses the CFS algorithm and is germane to any Linux kernel since 2.6.23. We discuss the scheduler class for real-time processes in a later section.

Process Scheduling in Unix Systems

To discuss fair scheduling, we must first describe how traditional Unix systems schedule processes. As mentioned in the previous section, modern process schedulers have two common concepts: process priority and timeslice. Timeslice is how long a process runs; processes start with some default timeslice. Processes with a higher priority run more frequently and (on many systems) receive a higher timeslice. On Unix, the priority is exported to user-space in the form of nice values. This sounds simple, but in practice it leads to several pathological problems, which we now discuss.

First, mapping nice values onto timeslices requires a decision about what absolute timeslice to allot each nice value. This leads to suboptimal switching behavior. For example, let's assume we assign processes of the default nice value (zero) a timeslice of 100 milliseconds and processes at the highest nice value (+20, the lowest priority) a timeslice of 5 milliseconds. Further, let's assume one of each of these processes is runnable. Our default-priority process thus receives 20/21 (100 out of 105 milliseconds) of the processor, whereas our low priority process receives 1/21 (5 out of 105 milliseconds) of the processor. We could have used any numbers for this example, but we assume this allotment is optimal since we chose it. Now, what happens if we run exactly two low priority processes? We'd expect they each receive 50% of the processor, which they do. But they each enjoy the processor for only 5 milliseconds at a time (5 out of 10 milliseconds each)! That is, instead of context switching twice every 105 milliseconds, we now context switch twice every 10 milliseconds. Conversely, if we have two normal priority processes, each again receives the correct 50% of the processor, but in 100 millisecond increments. Neither of these timeslice allotments are necessarily ideal; each is simply a byproduct of a given nice value to timeslice mapping coupled with a specific runnable process priority mix. Indeed, given that high nice value (low priority) processes tend to be background, processor-intensive tasks, while normal priority processes tend to be foreground user tasks, this timeslice allotment is exactly *backward* from ideal!

A second problem concerns relative nice values and again the nice value to timeslice mapping. Say we have two processes, each a single nice value apart. First, let's assume they are at nice values 0 and 1. This might map (and indeed did in the `O(1)` scheduler) to timeslices of 100 and 95 milliseconds, respectively. These two values are nearly identical, and thus the difference here between a single nice value is small. Now, instead, let's assume our two processes are at nice values of 18 and 19. This now maps to timeslices of

10 and 5 milliseconds, respectively—the former receiving twice the processor time as the latter! Because nice values are most commonly used in relative terms (as the system call accepts an increment, not an absolute value), this behavior means that “nicing down a process by one” has wildly different effects depending on the starting nice value.

Third, if performing a nice value to timeslice mapping, we need the ability to assign an absolute timeslice. This absolute value must be measured in terms the kernel can measure. In most operating systems, this means the timeslice must be some integer multiple of the timer tick. (See Chapter 11, “Timers and Time Management,” for a discussion on time.) This introduces several problems. First, the minimum timeslice has a floor of the period of the timer tick, which might be as high as 10 milliseconds or as low as 1 millisecond. Second, the system timer limits the difference between two timeslices; successive nice values might map to timeslices as much as 10 milliseconds or as little as 1 millisecond apart. Finally, timeslices change with different timer ticks. (If this paragraph’s discussion of timer ticks is foreign, reread it after reading Chapter 11. This is only one motivation behind CFS.)

The fourth and final problem concerns handling process wake up in a priority-based scheduler that wants to optimize for interactive tasks. In such a system, you might want to give freshly woken-up tasks a priority boost by allowing them to run immediately, even if their timeslice was expired. Although this improves interactive performance in many, if not most, situations, it also opens the door to pathological cases where certain sleep/wake up use cases can game the scheduler into providing one process an unfair amount of processor time, at the expense of the rest of the system.

Most of these problems are solvable by making substantial but not paradigm-shifting changes to the old-school Unix scheduler. For example, making nice values geometric instead of additive solves the second problem. And mapping nice values to timeslices using a measurement decoupled from the timer tick solves the third problem. But such solutions belie the true problem, which is that assigning absolute timeslices yields a constant switching rate but variable fairness. The approach taken by CFS is a radical (for process schedulers) rethinking of timeslice allotment: Do away with timeslices completely and assign each process a proportion of the processor. CFS thus yields constant fairness but a variable switching rate.

Fair Scheduling

CFS is based on a simple concept: Model process scheduling as if the system had an ideal, perfectly multitasking processor. In such a system, each process would receive $1/n$ of the processor’s time, where n is the number of runnable processes, and we’d schedule them for infinitely small durations, so that in any measurable period we’d have run all n processes for the same amount of time. As an example, assume we have two processes. In the standard Unix model, we might run one process for 5 milliseconds and then another process for 5 milliseconds. While running, each process would receive 100% of the processor. In an ideal, perfectly multitasking processor, we would run both processes *simultaneously* for 10 milliseconds, each at 50% power. This latter model is called *perfect multitasking*.

Of course, such a model is also impractical, because it is not possible on a single processor to *literally* run multiple processes simultaneously. Moreover, it is not efficient to run processes for infinitely small durations. That is, there is a *switching cost* to preempting one process for another: the overhead of swapping one process for another and the effects on caches, for example. Thus, although we'd like to run processes for very small durations, CFS is mindful of the overhead and performance hit in doing so. Instead, CFS will run each process for some amount of time, round-robin, selecting next the process that has run the least. Rather than assign each process a timeslice, CFS calculates how long a process should run as a function of the total number of runnable processes. Instead of using the nice value to calculate a timeslice, CFS uses the nice value to *weight* the proportion of processor a process is to receive: Higher valued (lower priority) processes receive a fractional weight relative to the default nice value, whereas lower valued (higher priority) processes receive a larger weight.

Each process then runs for a “timeslice” proportional to its weight divided by the total weight of all runnable threads. To calculate the actual timeslice, CFS sets a target for its approximation of the “infinitely small” scheduling duration in perfect multitasking. This target is called the *targeted latency*. Smaller targets yield better interactivity and a closer approximation to perfect multitasking, at the expense of higher switching costs and thus worse overall throughput. Let's assume the targeted latency is 20 milliseconds and we have two runnable tasks at the same priority. *Regardless of those task's priority*, each will run for 10 milliseconds before preempting in favor of the other. If we have four tasks at the same priority, each will run for 5 milliseconds. If there are 20 tasks, each will run for 1 millisecond.

Note as the number of runnable tasks approaches infinity, the proportion of allotted processor and the assigned timeslice approaches zero. As this will eventually result in unacceptable switching costs, CFS imposes a floor on the timeslice assigned to each process. This floor is called the *minimum granularity*. By default it is 1 millisecond. Thus, even as the number of runnable processes approaches infinity, each will run for at least 1 millisecond, to ensure there is a ceiling on the incurred switching costs. (Astute readers will note that CFS is thus not perfectly fair when the number of processes grows so large that the calculated proportion is floored by the minimum granularity. This is true. Although modifications to fair queuing exist to improve upon this fairness, CFS was explicitly designed to make this trade-off. In the common case of only a handful of runnable processes, CFS is perfectly fair.)

Now, let's again consider the case of two runnable processes, except with dissimilar nice values—say, one with the default nice value (zero) and one with a nice value of 5. These nice values have dissimilar weights and thus our two processes receive different proportions of the processor's time. In this case, the weights work out to about a 1/3 penalty for the nice-5 process. If our target latency is again 20 milliseconds, our two processes will receive 15 milliseconds and 5 milliseconds each of processor time, respectively. Say our two runnable processes instead had nice values of 10 and 15. What would be the allotted timeslices? Again 15 and 5 milliseconds each! Absolute nice values no

longer affect scheduling decisions; only relative values affect the proportion of processor time allotted.

Put generally, the proportion of processor time that any process receives is determined only by the relative difference in niceness between it and the other runnable processes. The nice values, instead of yielding additive increases to timeslices, yield geometric differences. The absolute timeslice allotted any nice value is not an absolute number, but a given proportion of the processor. CFS is called a *fair scheduler* because it gives each process a fair share—a proportion—of the processor's time. As mentioned, note that CFS isn't perfectly fair, because it only approximates perfect multitasking, but it *can* place a lower bound on latency of n for n runnable processes on the unfairness.

The Linux Scheduling Implementation

With the discussion of the motivation for and logic of CFS behind us, we can now explore CFS's actual implementation, which lives in `kernel/sched_fair.c`. Specifically, we discuss four components of CFS:

- Time Accounting
- Process Selection
- The Scheduler Entry Point
- Sleeping and Waking Up

Time Accounting

All process schedulers must account for the time that a process runs. Most Unix systems do so, as discussed earlier, by assigning each process a timeslice. On each tick of the system clock, the timeslice is decremented by the tick period. When the timeslice reaches zero, the process is preempted in favor of another runnable process with a nonzero timeslice.

The Scheduler Entity Structure

CFS does not have the notion of a timeslice, but it must still keep account for the time that each process runs, because it needs to ensure that each process runs only for its fair share of the processor. CFS uses the *scheduler entity structure*, `struct sched_entity`, defined in `<linux/sched.h>`, to keep track of process accounting:

```
struct sched_entity {
    struct load_weight    load;
    struct rb_node        run_node;
    struct list_head      group_node;
    unsigned int          on_rq;
    u64                   exec_start;
    u64                   sum_exec_runtime;
    u64                   vruntime;
    u64                   prev_sum_exec_runtime;
```

```

u64                last_wakeup;
u64                avg_overlap;
u64                nr_migrations;
u64                start_runtime;
u64                avg_wakeup;

/* many stat variables elided, enabled only if CONFIG_SCHEDSTATS is set */
};

```

The scheduler entity structure is embedded in the *process descriptor*, `struct task_struct`, as a member variable named `se`. We discussed the process descriptor in Chapter 3, “Process Management.”

The Virtual Runtime

The `vruntime` variable stores the *virtual runtime* of a process, which is the actual runtime (the amount of time spent running) normalized (or weighted) by the number of runnable processes. The virtual runtime’s units are nanoseconds and therefore `vruntime` is decoupled from the timer tick. The virtual runtime is used to help us approximate the “ideal multitasking processor” that CFS is modeling. With such an ideal processor, we wouldn’t need `vruntime`, because all runnable processes would perfectly multitask. That is, on an ideal processor, the virtual runtime of all processes of the same priority would be identical—all tasks would have received an equal, fair share of the processor. Because processors are not capable of perfect multitasking and we must run each process in succession, CFS uses `vruntime` to account for how long a process has run and thus how much longer it ought to run.

The function `update_curr()`, defined in `kernel/sched_fair.c`, manages this accounting:

```

static void update_curr(struct cfs_rq *cfs_rq)
{
    struct sched_entity *curr = cfs_rq->curr;
    u64 now = rq_of(cfs_rq)->clock;
    unsigned long delta_exec;

    if (unlikely(!curr))
        return;

    /*
     * Get the amount of time the current task was running
     * since the last time we changed load (this cannot
     * overflow on 32 bits):
     */
    delta_exec = (unsigned long)(now - curr->exec_start);
    if (!delta_exec)
        return;
}

```

```

__update_curr(cfs_rq, curr, delta_exec);
curr->exec_start = now;

if (entity_is_task(curr)) {
    struct task_struct *curtask = task_of(curr);

    trace_sched_stat_runtime(curtask, delta_exec, curr->vruntime);
    cpuacct_charge(curtask, delta_exec);
    account_group_exec_runtime(curtask, delta_exec);
}
}

```

`update_curr()` calculates the execution time of the current process and stores that value in `delta_exec`. It then passes that runtime to `__update_curr()`, which weights the time by the number of runnable processes. The current process's `vruntime` is then incremented by the weighted value:

```

/*
 * Update the current task's runtime statistics. Skip current tasks that
 * are not in our scheduling class.
 */
static inline void
__update_curr(struct cfs_rq *cfs_rq, struct sched_entity *curr,
              unsigned long delta_exec)
{
    unsigned long delta_exec_weighted;

    schedstat_set(curr->exec_max, max((u64)delta_exec, curr->exec_max));

    curr->sum_exec_runtime += delta_exec;
    schedstat_add(cfs_rq, exec_clock, delta_exec);
    delta_exec_weighted = calc_delta_fair(delta_exec, curr);

    curr->vruntime += delta_exec_weighted;
    update_min_vruntime(cfs_rq);
}

```

`update_curr()` is invoked periodically by the system timer and also whenever a process becomes runnable or blocks, becoming unrunnable. In this manner, `vruntime` is an accurate measure of the runtime of a given process and an indicator of what process should run next.

Process Selection

In the last section, we discussed how `vruntime` on an ideal, perfectly multitasking processor would be identical among all runnable processes. In reality, we cannot perfectly multitask, so CFS attempts to balance a process's virtual runtime with a simple rule: When CFS

is deciding what process to run next, it picks the process with the smallest `vruntime`. This is, in fact, the core of CFS's scheduling algorithm: Pick the task with the smallest `vruntime`. That's it! The rest of this subsection describes how the selection of the process with the smallest `vruntime` is implemented.

CFS uses a *red-black tree* to manage the list of runnable processes and efficiently find the process with the smallest `vruntime`. A red-black tree, called an *rbtree* in Linux, is a type of *self-balancing binary search tree*. We discuss self-balancing binary search trees in general and red-black trees in particular in Chapter 6. For now, if you are unfamiliar, you need to know only that red-black trees are a data structure that store *nodes* of arbitrary data, identified by a specific *key*, and that they enable efficient search for a given *key*. (Specifically, obtaining a node identified by a given key is logarithmic in time as a function of total nodes in the tree.)

Picking the Next Task

Let's start with the assumption that we have a red-black tree populated with every runnable process in the system where the key for each node is the runnable process's virtual runtime. We'll look at how we build that tree in a moment, but for now let's assume we have it. Given this tree, the process that CFS wants to run next, which is the process with the smallest `vruntime`, is the leftmost node in the tree. That is, if you follow the tree from the root down through the left child, and continue moving to the left until you reach a leaf node, you find the process with the smallest `vruntime`. (Again, if you are unfamiliar with binary search trees, don't worry. Just know that this process is efficient.) CFS's process selection algorithm is thus summed up as "run the process represented by the leftmost node in the *rbtree*." The function that performs this selection is

`__pick_next_entity()`, defined in `kernel/sched_fair.c`:

```
static struct sched_entity *__pick_next_entity(struct cfs_rq *cfs_rq)
{
    struct rb_node *left = cfs_rq->rb_leftmost;

    if (!left)
        return NULL;

    return rb_entry(left, struct sched_entity, run_node);
}
```

Note that `__pick_next_entity()` does not actually traverse the tree to find the leftmost node, because the value is cached by `rb_leftmost`. Although it is efficient to walk the tree to find the leftmost node— $O(\text{height of tree})$, which is $O(\log N)$ for N nodes if the tree is balanced—it is even easier to cache the leftmost node. The return value from this function is the process that CFS next runs. If the function returns `NULL`, there is no leftmost node, and thus no nodes in the tree. In that case, there are no runnable processes, and CFS schedules the idle task.

Adding Processes to the Tree

Now let's look at how CFS adds processes to the rbtree and caches the leftmost node. This would occur when a process becomes runnable (wakes up) or is first created via `fork()`, as discussed in Chapter 3. Adding processes to the tree is performed by `enqueue_entity()`:

```
static void
enqueue_entity(struct cfs_rq *cfs_rq, struct sched_entity *se, int flags)
{
    /*
     * Update the normalized vruntime before updating min_vruntime
     * through callig update_curr().
     */
    if (!(flags & ENQUEUE_WAKEUP) || (flags & ENQUEUE_MIGRATE))
        se->vruntime += cfs_rq->min_vruntime;

    /*
     * Update run-time statistics of the 'current'.
     */
    update_curr(cfs_rq);
    account_entity_enqueue(cfs_rq, se);

    if (flags & ENQUEUE_WAKEUP) {
        place_entity(cfs_rq, se, 0);
        enqueue_sleeper(cfs_rq, se);
    }

    update_stats_enqueue(cfs_rq, se);
    check_spread(cfs_rq, se);
    if (se != cfs_rq->curr)
        __enqueue_entity(cfs_rq, se);
}
```

This function updates the runtime and other statistics and then invokes `__enqueue_entity()` to perform the actual heavy lifting of inserting the entry into the red-black tree:

```
/*
 * Enqueue an entity into the rb-tree:
 */
static void __enqueue_entity(struct cfs_rq *cfs_rq, struct sched_entity *se)
{
    struct rb_node **link = &cfs_rq->tasks_timeline.rb_node;
    struct rb_node *parent = NULL;
    struct sched_entity *entry;
    s64 key = entity_key(cfs_rq, se);
    int leftmost = 1;
```

```

/*
 * Find the right place in the rbtree:
 */
while (*link) {
    parent = *link;
    entry = rb_entry(parent, struct sched_entity, run_node);
    /*
     * We dont care about collisions. Nodes with
     * the same key stay together.
     */
    if (key < entity_key(cfs_rq, entry)) {
        link = &parent->rb_left;
    } else {
        link = &parent->rb_right;
        leftmost = 0;
    }
}

/*
 * Maintain a cache of leftmost tree entries (it is frequently
 * used):
 */
if (leftmost)
    cfs_rq->rb_leftmost = &se->run_node;

rb_link_node(&se->run_node, parent, link);
rb_insert_color(&se->run_node, &cfs_rq->tasks_timeline);
}

```

Let's look at this function. The body of the `while()` loop traverses the tree in search of a matching key, which is the inserted process's `vruntime`. Per the rules of the balanced tree, it moves to the left child if the key is smaller than the current node's key and to the right child if the key is larger. If it ever moves to the right, even once, it knows the inserted process cannot be the new leftmost node, and it sets `leftmost` to zero. If it moves only to the left, `leftmost` remains one, and we have a new leftmost node and can update the cache by setting `rb_leftmost` to the inserted process. The loop terminates when we compare ourselves to a node that has no child in the direction we move; `link` is then `NULL` and the loop terminates. When out of the loop, the function calls `rb_link_node()` on the parent node, making the inserted process the new child. The function `rb_insert_color()` updates the self-balancing properties of the tree; we discuss the coloring in Chapter 6.

Removing Processes from the Tree

Finally, let's look at how CFS removes processes from the red-black tree. This happens when a process blocks (becomes unrunnable) or terminates (ceases to exist):

```
static void
dequeue_entity(struct cfs_rq *cfs_rq, struct sched_entity *se, int sleep)
{
    /*
     * Update run-time statistics of the 'current'.
     */
    update_curr(cfs_rq);

    update_stats_dequeue(cfs_rq, se);
    clear_buddies(cfs_rq, se);

    if (se != cfs_rq->curr)
        __dequeue_entity(cfs_rq, se);
    account_entity_dequeue(cfs_rq, se);
    update_min_vruntime(cfs_rq);

    /*
     * Normalize the entity after updating the min_vruntime because the
     * update can refer to the ->curr item and we need to reflect this
     * movement in our normalized position.
     */
    if (!sleep)
        se->vruntime -= cfs_rq->min_vruntime;
}
```

As with adding a process to the red-black tree, the real work is performed by a helper function, `__dequeue_entity()`:

```
static void __dequeue_entity(struct cfs_rq *cfs_rq, struct sched_entity *se)
{
    if (cfs_rq->rb_leftmost == &se->run_node) {
        struct rb_node *next_node;

        next_node = rb_next(&se->run_node);
        cfs_rq->rb_leftmost = next_node;
    }

    rb_erase(&se->run_node, &cfs_rq->tasks_timeline);
}
```

Removing a process from the tree is much simpler because the rbtree implementation provides the `rb_erase()` function that performs all the work. The rest of this function updates the `rb_leftmost` cache. If the process-to-remove is the leftmost node, the func-

tion invokes `rb_next()` to find what would be the next node in an in-order traversal. This is what will be the leftmost node when the current leftmost node is removed.

The Scheduler Entry Point

The main entry point into the process schedule is the function `schedule()`, defined in `kernel/sched.c`. This is the function that the rest of the kernel uses to invoke the process scheduler, deciding which process to run and then running it. `schedule()` is generic with respect to scheduler classes. That is, it finds the highest priority scheduler class with a runnable process and asks it what to run next. Given that, it should be no surprise that `schedule()` is simple. The only important part of the function—which is otherwise too uninteresting to reproduce here—is its invocation of `pick_next_task()`, also defined in `kernel/sched.c`. The `pick_next_task()` function goes through each scheduler class, starting with the highest priority, and selects the highest priority process in the highest priority class:

```
/*
 * Pick up the highest-prio task:
 */
static inline struct task_struct *
pick_next_task(struct rq *rq)
{
    const struct sched_class *class;
    struct task_struct *p;

    /*
     * Optimization: we know that if all tasks are in
     * the fair class we can call that function directly:
     */
    if (likely(rq->nr_running == rq->cfs.nr_running)) {
        p = fair_sched_class.pick_next_task(rq);
        if (likely(p))
            return p;
    }

    class = sched_class_highest;
    for ( ; ; ) {
        p = class->pick_next_task(rq);
        if (p)
            return p;

        /*
         * Will never be NULL as the idle class always
         * returns a non-NULL p:
         */
        class = class->next;
    }
}
```

Note the optimization at the beginning of the function. Because CFS is the scheduler class for normal processes, and most systems run mostly normal processes, there is a small hack to quickly select the next CFS-provided process if the number of runnable processes is equal to the number of CFS runnable processes (which suggests that all runnable processes are provided by CFS).

The core of the function is the `for()` loop, which iterates over each class in priority order, starting with the highest priority class. Each class implements the `pick_next_task()` function, which returns a pointer to its next runnable process or, if there is not one, `NULL`. The first class to return a non-`NULL` value has selected the next runnable process. CFS's implementation of `pick_next_task()` calls `pick_next_entity()`, which in turn calls the `__pick_next_entity()` function that we discussed in the previous section.

Sleeping and Waking Up

Tasks that are sleeping (blocked) are in a special nonrunnable state. This is important because without this special state, the scheduler would select tasks that did not want to run or, worse, sleeping would have to be implemented as busy looping. A task sleeps for a number of reasons, but always while it is waiting for some event. The event can be a specified amount of time, more data from a file I/O, or another hardware event. A task can also involuntarily go to sleep when it tries to obtain a contended semaphore in the kernel (this is covered in Chapter 9, “An Introduction to Kernel Synchronization”). A common reason to sleep is file I/O—for example, the task issued a `read()` request on a file, which needs to be read in from disk. As another example, the task could be waiting for keyboard input. Whatever the case, the kernel behavior is the same: The task marks itself as sleeping, puts itself on a wait queue, removes itself from the red-black tree of runnable, and calls `schedule()` to select a new process to execute. Waking back up is the inverse: The task is set as runnable, removed from the wait queue, and added back to the red-black tree.

As discussed in the previous chapter, two states are associated with sleeping, `TASK_INTERRUPTIBLE` and `TASK_UNINTERRUPTIBLE`. They differ only in that tasks in the `TASK_UNINTERRUPTIBLE` state ignore signals, whereas tasks in the `TASK_INTERRUPTIBLE` state wake up prematurely and respond to a signal if one is issued. Both types of sleeping tasks sit on a wait queue, waiting for an event to occur, and are not runnable.

Wait Queues

Sleeping is handled via wait queues. A wait queue is a simple list of processes waiting for an event to occur. Wait queues are represented in the kernel by `wake_queue_head_t`. Wait queues are created statically via `DECLARE_WAITQUEUE()` or dynamically via `init_waitqueue_head()`. Processes put themselves on a wait queue and mark themselves not runnable. When the event associated with the wait queue occurs, the processes on the queue are awakened. It is important to implement sleeping and waking correctly, to avoid race conditions.